온리티칭 ERD 설계서

작성일: 2026-04-08
참고: 배달의민족 ERD 구조

---

0. 배민 ERD에서 배운 핵심 원칙

배민 ERD를 분석하면 다음 패턴이 일관됩니다.

원칙	배민 사례	우리에게 주는 시사점
공통 컬럼 표준화	모든 테이블에 `createdDate / modifiedDate / status`	모든 테이블에 동일 패턴 — 감사·복구·소프트삭제 일원화
한 도메인 = 한 테이블	주문/장바구니/메뉴/메뉴옵션 분리	도메인 책임 분리, JOIN으로 조립
이미지·텍스트는 별도 테이블	StoreImage 분리	큰 컬럼이 메인 테이블 row 크기 부풀리는 것 방지
카운트 비정규화	Stores.dibsCount, reviewCount 컬럼	매번 COUNT(*) 안 돌리고 컬럼만 읽기
상태(status) varchar	'NORMAL', 'DELETED' 등	enum 변경 자유, 인덱스 가능
bigint(20) PK	모든 PK	사용자 폭증 대비
조인 테이블 명확화	Carts (userId+storeId+menuId+orderId)	다대다 관계는 별도 테이블
soft delete	status='DELETED'	삭제 안 함, 복구 가능

---

1. 온리티칭 도메인 분석

1-1. 도메인 분리

도메인	설명
Auth	학교/교사 인증
School	학교, 학년, 반
Student	학생 명부
Care	돌봄교실 (감정·투두·이벤트)
Schedule	학사일정
Records	생활기록부, 관찰일지
Evaluation	교과평가, 성취기준
Activity	창의적 체험활동
Meal	급식
Newsletter	가정통신문
Tools	발표자/자리/1인1역 (사용 로그만)
Autobiography	자서전 (다른 도메인 데이터 읽기 전용)

1-2. 배민 매핑 비교

배민	온리티칭
Users (회원)	Teachers (교사)
Stores (가게)	Schools (학교)
Menu (메뉴)	Subjects (과목)
Orders (주문)	LifeRecords (생활기록부 작성)
Carts (장바구니)	LifeRecordDrafts (작성중인 코멘트)
Reviews (리뷰)	ObservationLogs (관찰일지)
Coupons (쿠폰)	Permissions (권한)
StoreImage (가게사진)	StudentPhoto (학생사진)

---

2. 핵심 테이블 설계

2-1. 공통 컬럼 (모든 테이블)

```sql
id          BIGINT PK AUTO_INCREMENT
```

```
createdDate   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
modifiedDate  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
status        VARCHAR(20) NOT NULL DEFAULT 'NORMAL'
```

2-2. Auth & School 도메인

schools (학교)

```sql
schoolId       BIGINT PK
name           VARCHAR(100) NOT NULL    -- "○○초등학교"
schoolCode     VARCHAR(20) UNIQUE NOT NULL  -- 교육청 학교 코드
region         VARCHAR(50)              -- 시도교육청
address        VARCHAR(255)
phone          VARCHAR(20)
+ 공통 컬럼
```

teachers (교사 = 회원)

```sql
teacherId      BIGINT PK
schoolId       BIGINT FK → schools
loginId        VARCHAR(50) UNIQUE NOT NULL  -- 로그인 ID
password       VARCHAR(255) NOT NULL        -- bcrypt
name           VARCHAR(50) NOT NULL
role           VARCHAR(20) NOT NULL DEFAULT 'TEACHER'
               -- TEACHER / VICE_PRINCIPAL / PRINCIPAL / ADMIN
subject        VARCHAR(50)              -- 전담교사일 때 과목
phone          VARCHAR(20)
email          VARCHAR(100)
lastLoginAt    TIMESTAMP
+ 공통 컬럼
```

classes (학급)

```sql
classId        BIGINT PK
schoolId       BIGINT FK → schools
teacherId      BIGINT FK → teachers     -- 담임
year           INT NOT NULL             -- 2026
grade          INT NOT NULL             -- 1~6
classNo        INT NOT NULL             -- 반 번호
+ 공통 컬럼
UNIQUE(schoolId, year, grade, classNo)
```

2-3. Student 도메인

students (학생)

```sql
studentId      BIGINT PK
classId        BIGINT FK → classes
number         INT NOT NULL             -- 출석번호
name           VARCHAR(50) NOT NULL
gender         CHAR(1)                  -- M/F
birthDate      DATE
phone          VARCHAR(20)
guardianName   VARCHAR(50)
guardianPhone  VARCHAR(20)
address        VARCHAR(255)
+ 공통 컬럼
INDEX(classId, number)
```

2-4. Care 도메인 (돌봄교실 — 자서전 핵심)

care_records (돌봄교실 일일 기록)
```sql
careRecordId    BIGINT PK
teacherId       BIGINT FK → teachers
classId         BIGINT FK → classes
recordDate      DATE NOT NULL
overallMood     VARCHAR(50)             -- 전체 분위기
importantEvents TEXT                    -- 자유 입력 (특이사항)
+ 공통 컬럼
UNIQUE(teacherId, recordDate)
INDEX(teacherId, recordDate DESC)       -- 자서전 조회 최적화
```

care_todos (돌봄 투두)
```sql
todoId          BIGINT PK
careRecordId    BIGINT FK → care_records
content         VARCHAR(255) NOT NULL
isDeadline      TINYINT(1) DEFAULT 0
isDone          TINYINT(1) DEFAULT 0
sortOrder       INT DEFAULT 0
+ 공통 컬럼
INDEX(careRecordId)
```

care_student_moods (학생별 감정 기록)
```sql
moodId          BIGINT PK
careRecordId    BIGINT FK → care_records
studentId       BIGINT FK → students
mood            VARCHAR(50) NOT NULL    -- 'happy', 'sad', ...
customMood      VARCHAR(100)            -- 직접 입력
note            VARCHAR(255)
+ 공통 컬럼
INDEX(careRecordId, studentId)
INDEX(studentId, createdDate DESC)      -- 학생별 자서전용
```

2-5. Schedule 도메인

schedules (학사일정)
```sql
scheduleId      BIGINT PK
schoolId        BIGINT FK → schools
title           VARCHAR(100) NOT NULL
description     TEXT
startDate       DATE NOT NULL
endDate         DATE
isMajorEvent    TINYINT(1) DEFAULT 0    -- 큰 행사 플래그 ★자서전 사건순용
eventType       VARCHAR(30)             -- 'SPORT', 'TRIP', 'EXAM', ...
+ 공통 컬럼
INDEX(schoolId, startDate)
INDEX(schoolId, isMajorEvent, startDate)
```

2-6. Records 도메인

life_records (생활기록부 코멘트)
```sql
lifeRecordId    BIGINT PK
studentId       BIGINT FK → students
teacherId       BIGINT FK → teachers
keyword         VARCHAR(50) NOT NULL    -- '발표능력', '협력', ...
content         TEXT NOT NULL
```

```
semester        INT                     -- 1, 2
year            INT
+ 공통 컬럼
INDEX(studentId, year, semester)
INDEX(teacherId, createdDate DESC)
```

observation_logs (관찰일지)
```sql
logId           BIGINT PK
studentId       BIGINT FK → students
teacherId       BIGINT FK → teachers
observedDate    DATE NOT NULL
category        VARCHAR(30)             -- 'BEHAVIOR', 'EMOTION', ...
content         TEXT NOT NULL
+ 공통 컬럼
INDEX(studentId, observedDate DESC)
```

2-7. 자서전 도메인 (배민의 Order ↔ 우리)

autobiography_jobs (자서전 생성 작업)
```sql
jobId           BIGINT PK
teacherId       BIGINT FK → teachers
mode            VARCHAR(20) NOT NULL     -- 'TEACHER', 'STUDENT', 'PRINCIPAL', 'VICE'
outlineMode     VARCHAR(20)             -- 'CHRONO', 'EVENT', 'STUDENT', 'THEMATIC'
prompt          TEXT
sourceData      JSON                    -- 비정규화 — 생성 시점 데이터 스냅샷
generatedText   LONGTEXT
status          VARCHAR(20) DEFAULT 'PENDING'  -- PENDING/PROCESSING/DONE/FAILED
processedAt     TIMESTAMP
+ 공통 컬럼
INDEX(teacherId, createdDate DESC)
INDEX(status, createdDate)              -- 워커 큐 조회용
```

2-8. Tools 도메인 (수업 보조 도구)

tool_logs (도구 사용 로그)
```sql
logId           BIGINT PK
teacherId       BIGINT FK → teachers
toolType        VARCHAR(30) NOT NULL     -- 'PRESENTER', 'SEAT', 'ROLE'
inputData       JSON                    -- 학생 풀, 자리 수 등
outputData      JSON                    -- 결과
+ 공통 컬럼
INDEX(teacherId, createdDate DESC)
```

---

3. 병목 방지 핵심 전략 7가지

3-1. ★ 인덱스 (가장 중요)

```sql
-- 자서전 생성 시 가장 자주 쓰이는 쿼리
CREATE INDEX idx_care_teacher_date ON care_records(teacherId, recordDate DESC);
CREATE INDEX idx_mood_student_date ON care_student_moods(studentId, createdDate DESC);
CREATE INDEX idx_schedule_event ON schedules(schoolId, isMajorEvent, startDate);
CREATE INDEX idx_life_student_year ON life_records(studentId, year, semester);

-- 로그인
CREATE UNIQUE INDEX idx_teacher_login ON teachers(loginId);
```

```
-- 다중 컬럼 검색
CREATE INDEX idx_class_school_year ON classes(schoolId, year, grade, classNo);
```

인덱스 1개 = 응답속도 100배 차이. 사용자 1만명 시 인덱스 없으면 로그인 1건당 1초+, 있으면 1ms.

3-2. ★ 비정규화 (배민의 진짜 핵심)

사례 1 — 카운트 컬럼
```
care_records 테이블에 todoCount, doneTodoCount 컬럼 추가
→ 매번 SELECT COUNT(*) FROM care_todos 안 돌림
→ todo 추가/삭제 시 트리거 또는 애플리케이션에서 ±1
```

사례 2 — 자서전 sourceData 스냅샷
```
autobiography_jobs.sourceData JSON에 생성 시점 데이터를 통째로 저장
→ 학생이 전학 가도, 일정이 수정돼도 그 자서전은 영원히 동일
→ 재생성 시 JOIN 0회
```

사례 3 — 학생 이름 복사
```
care_student_moods 테이블에 studentName VARCHAR(50) 컬럼 추가
→ students 테이블 JOIN 없이 바로 조회
→ 학생 이름 바뀌면 트리거로 동기화 OR 그냥 안 바꿈 (자서전 기록 보존)
```

3-3. 읽기/쓰기 분리 (Read Replica)

```
[App Server] ─쓰기→ [Primary DB]
            ↓              ↓ replication (1초 지연)
            ↘──────읽기──────→ [Read Replica]

자서전 생성, 학사일정 조회, 학생 명부 조회 → Replica
로그인, 데이터 저장 → Primary
```

배민도 주문 조회는 100% Replica.

3-4. 캐싱 (Redis)

캐시 대상	TTL	효과
학사일정 (1년)	1일	DB 조회 거의 0
학생 명부 (반별)	1시간	자서전 생성 시 즉시 조회
자서전 결과	영구 (입력 해시 키)	같은 자서전 재생성 0초
사용자 세션	30일	DB 부담 없음
성취기준 마스터	7일	정적 데이터

3-5. 페이지네이션 강제

```sql
-- ❌ 절대 금지
SELECT * FROM care_records WHERE teacherId = ?;

-- ✅ 항상
SELECT * FROM care_records WHERE teacherId = ?
  ORDER BY recordDate DESC LIMIT 30 OFFSET 0;
```

3-6. 비동기 큐 (자서전 생성)

```
사용자 클릭
  ↓
API: jobId 즉시 반환 (0.1초)
  ↓
BullMQ 큐에 enqueue
  ↓
백그라운드 워커가 처리 (5~30초)
  ↓
완료되면 SSE/WebSocket으로 알림
  ↓
프론트가 결과 표시
```

장점:
- API 서버 동시 요청 막힘 0
- 사용자는 기다리는 동안 다른 작업
- 워커만 수평 확장하면 됨

3-7. 파일/이미지 분리

배민이 StoreImage를 별도 테이블로 뺀 이유와 동일.

```
✅ S3 / Cloudflare R2에 업로드
✅ DB에는 URL만 저장 (text)
❌ DB에 BLOB 저장 절대 금지
```

학생 사진, 급식 사진 등 이미지는 무조건 외부 스토리지.

---

4. 사용자 규모별 로드맵

사용자 수	필수 작업
~100명	인덱스 (3-1) + 페이지네이션 (3-5)
~1,000명	캐싱 (3-4) + 비정규화 카운트 (3-2)
~10,000명	Read Replica (3-3) + 자서전 비동기 큐 (3-6)
~100,000명	파일 외부 스토리지 (3-7) + DB 샤딩 검토
~1,000,000명	도메인별 마이크로서비스 분리

---

5. ERD 다이어그램 (텍스트 버전)

```
schools (학교)
   │
   ├──< teachers (교사) ──< autobiography_jobs (자서전)
   │       │
   │       └──< tool_logs
   │
   └──< classes (학급)
          │
          └──< students (학생)
                 │
                 ├──< life_records (생활기록부)
                 └──< observation_logs (관찰일지)

teachers ──< care_records (돌봄교실 일일)
```

```
                │
                ├──── < care_todos
                └──── < care_student_moods >──── students

schools ──── < schedules (학사일정)
```

---

6. 즉시 적용 우선순위 (To-Do)

우선순위	작업	효과
🔴 P0	모든 테이블 공통 컬럼 표준화 (createdDate/modifiedDate/status)	운영 안정성 확보
🔴 P0	care_records / care_todos / care_student_moods 분리	자서전 시간순 조회 최적화
🔴 P0	핵심 인덱스 6개 추가 (3-1 항목)	응답속도 100배
🟡 P1	autobiography_jobs.sourceData 비정규화	자서전 생성 안정성
🟡 P1	자서전 비동기 큐 (BullMQ)	동시 요청 시 멈춤 방지
🟢 P2	Redis 캐싱 (학사일정/학생명부)	사용자 1000명+ 대비
🟢 P2	이미지 S3 분리	DB 비대화 방지

---

7. 마무리

배민 ERD는 화려하지 않지만 **"기본을 지키는 것"** 이 핵심입니다.
- 공통 컬럼 표준화
- 도메인 분리
- 인덱스
- 비정규화 (필요한 곳에만)
- 카운트는 컬럼으로
- soft delete

이 6가지만 지키면 사용자 1만명까지는 무리 없이 갑니다.
그 이후에는 Read Replica + 캐싱 + 비동기 큐로 10만명까지.